

# Tabela de Mensagens — Firmware STM32G070 v3

## Protocolo de quadro:

- **RX** (Raspberry Pi → STM32): `[0xAA][CMD]` ou `[0xAA][CMD][DATA]`
- **TX** (STM32 → Raspberry Pi): `[0x90][PAYLOAD]` (sucesso), `[0x90][CMD][DATA]` (telemetria), ou `[0x91]` (erro)

## Tabela 1 — Dicionário Completo de Mensagens

| Símbolo         | Byte | Direção | Tipo      | Descrição                                                                                                                                                                                          |
|-----------------|------|---------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| START_FRAME_MSG | 0xAA | RX      | Estrutura | Byte de início de quadro. Todo frame recebido deve começar com este byte; bytes fora de quadro são ignorados silenciosamente.                                                                      |
| CMD_OK_MSG      | 0x90 | TX      | Estrutura | Byte de status positivo. Enviado como primeiro byte de qualquer resposta bem-sucedida, sempre seguido de um payload.                                                                               |
| CMD_ERR_MSG     | 0x91 | TX      | Estrutura | Byte de status negativo. Enviado sozinho (1 byte) quando o comando recebido não é reconhecido.                                                                                                     |
| SYS_RDY_MSG     | 0x10 | RX      | Handshake | Mensagem enviada pelo Raspberry Pi para iniciar o handshake. Só é aceita após todos os quatro sensores estarem operando na frequência esperada (200 Hz).                                           |
| SYS_INIT_MSG    | 0x01 | TX      | Handshake | Payload de resposta ao handshake. Indica que o STM32 está inicializado. Enviado no quadro <code>[0x90][0x01]</code> , sempre seguido de <code>[0x90][0x11]</code> informando o modo inicial.       |
| MODE_FSM_MSG    | 0x11 | TX      | Modo      | Payload que informa que o modo ativo é <code>OP_MODE_FSM</code> . Enviado após handshake e após cada transição de modo.                                                                            |
| MODE_DEBUG_MSG  | 0x22 | TX      | Modo      | Payload que informa que o modo ativo é <code>OP_MODE_DEBUG</code> . Enviado após cada transição de modo.                                                                                           |
| SW_RESET_MSG    | 0x33 | RX      | Controle  | Comando de reset por software. Processado com prioridade máxima (antes do toggle de modo e da FSM). O STM32 envia <code>[0x90][0x33]</code> e executa <code>NVIC_SystemReset()</code> após ~50 ms. |

| Símbolo                 | Byte | Direção | Tipo       | Descrição                                                                                                                                                                                                                                                                                                          |
|-------------------------|------|---------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ROUTE_A_RECEIVE_MSG     | 0xDA | RX      | FSM        | Comando enviado pelo Raspberry Pi ordenando o encaminhamento do objeto para a Rota A. Válido apenas no estado <b>STATE_WAIT_CLASSIFICATION</b> .                                                                                                                                                                   |
| ROUTE_B_RECEIVE_MSG     | 0xDB | RX      | FSM        | Comando enviado pelo Raspberry Pi ordenando o encaminhamento do objeto para a Rota B. Válido apenas no estado <b>STATE_WAIT_CLASSIFICATION</b> .                                                                                                                                                                   |
| OBJ_DETECTED_MSG        | 0xA0 | TX      | FSM        | Notificação espontânea de que o Sensor 1 detectou um objeto na entrada da esteira (transição <b>IDLE</b> → <b>OBJECT_DETECTED</b> ).                                                                                                                                                                               |
| CLSS_REQUEST_MSG        | 0xC0 | TX      | FSM        | Solicitação de classificação enviada ao Raspberry Pi quando o objeto alcança o Sensor 2 (transição <b>OBJECT_DETECTED</b> → <b>WAIT_CLASSIFICATION</b> ). O STM32 aguarda 0xDA ou 0xDB em resposta.                                                                                                                |
| ROUTE_A_FWRDNG_MSG      | 0xFA | TX      | FSM        | Confirmação de que o objeto está sendo encaminhado para a Rota A. Enviado ao receber 0xDA.                                                                                                                                                                                                                         |
| ROUTE_B_FWRDNG_MSG      | 0xFB | TX      | FSM        | Confirmação de que o objeto está sendo encaminhado para a Rota B. Enviado ao receber 0xDB.                                                                                                                                                                                                                         |
| ROUTE_A_SCCSS_DLVRV_MSG | 0xBA | TX      | FSM        | Notificação de entrega bem-sucedida na Rota A. Disparado quando o Sensor 3 detecta a passagem do objeto. FSM retorna a <b>STATE_IDLE</b> .                                                                                                                                                                         |
| ROUTE_B_SCCSS_DLVRV_MSG | 0xBB | TX      | FSM        | Notificação de entrega bem-sucedida na Rota B. Disparado quando o Sensor 4 detecta a passagem do objeto. FSM retorna a <b>STATE_IDLE</b> .                                                                                                                                                                         |
| DEBUG_MODE_TOGGLE_MSG   | 0xDD | RX      | Modo       | Alterna entre <b>OP_MODE_FSM</b> e <b>OP_MODE_DEBUG</b> . Processado com prioridade máxima (após <b>SW_RESET_MSG</b> , antes da FSM) em qualquer modo de operação.                                                                                                                                                 |
| SENS_STATUS_MSG         | 0x55 | TX      | Telemetria | Mensagem espontânea de telemetria dos sensores. Enviada apenas em <b>OP_MODE_DEBUG</b> , somente quando o STATUS_BYTE muda. Frame de 3 bytes: [0x90][0x55][STATUS_BYTE]. Bits 0–3 do STATUS_BYTE indicam o estado de SENS1–SENS4 respectivamente (1 = objeto detectado, 0 = feixe livre). Bits 4–7 reservados (0). |

| Símbolo               | Byte | Direção | Tipo               | Descrição                                                                                                                                                                                                                             |
|-----------------------|------|---------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LIGHT_EN_MSG          | 0xE1 | RX      | Assíncrono (Debug) | Liga o flash/luminária (FLASH_PIN → HIGH). Válido apenas em OP_MODE_DEBUG.                                                                                                                                                            |
| LIGHT_DISABLE_MSG     | 0xD1 | RX      | Assíncrono (Debug) | Desliga o flash/luminária (FLASH_PIN → LOW). Válido apenas em OP_MODE_DEBUG.                                                                                                                                                          |
| GATE_OPEN_MSG         | 0xE2 | RX      | Assíncrono (Debug) | Abre a cancela movendo o servo para <code>openAngle</code> (45°). Válido apenas em OP_MODE_DEBUG.                                                                                                                                     |
| GATE_CLOSE_MSG        | 0xD2 | RX      | Assíncrono (Debug) | Fecha a cancela movendo o servo para <code>closeAngle</code> (0°). Válido apenas em OP_MODE_DEBUG.                                                                                                                                    |
| STPR_EN_MSG           | 0xE3 | RX      | Assíncrono (Debug) | Ativa o motor de passo no modo <i>follow steps</i> (eixo engajado, TIM6 habilitado). Válido apenas em OP_MODE_DEBUG.                                                                                                                  |
| STPR_DISABLE_MSG      | 0xD3 | RX      | Assíncrono (Debug) | Desativa o motor de passo com eixo livre (driver A4988 em SLEEP). Válido apenas em OP_MODE_DEBUG.                                                                                                                                     |
| SET_STPR_FORWARD_MSG  | 0xE4 | RX      | Assíncrono (Debug) | Define o sentido de rotação do motor de passo como <b>frente</b> ( <code>stepperDirInst = 0</code> ). Válido apenas em OP_MODE_DEBUG.                                                                                                 |
| SET_STPR_BACKWARD_MSG | 0xD4 | RX      | Assíncrono (Debug) | Define o sentido de rotação do motor de passo como <b>trás</b> ( <code>stepperDirInst = 1</code> ). Válido apenas em OP_MODE_DEBUG.                                                                                                   |
| SET_STPR_TGT_STPS_MSG | 0xE5 | RX      | Assíncrono (Debug) | Inicia um quadro de 3 bytes. O byte <code>DATA</code> subsequente define a quantidade de passos alvo ( <code>targetSteps</code> ). É o único frame RX de 3 bytes ( <code>[0xAA][0xE5][DATA]</code> ). Válido apenas em OP_MODE_DEBUG. |

Tabela 2 — Fluxo de Mensagens por Evento

Legenda de quadros completos:

- Quadro TX de sucesso (2 bytes): `[0x90][PAYLOAD]`
- Quadro TX de telemetria (3 bytes): `[0x90][0x55][STATUS_BYTE]`
- Quadro TX de erro (1 byte): `[0x91]`
- Quadro RX padrão (2 bytes): `[0xAA][CMD]`
- Quadro RX com dado (3 bytes): `[0xAA][0xE5][DATA]`

Modo Normal — OP\_MODE\_FSM

Sequência de Inicialização (Handshake)

| # | Evento                                  | Agente      | Quadro Completo        | Bytes (hex) | Descrição                                                                                     |
|---|-----------------------------------------|-------------|------------------------|-------------|-----------------------------------------------------------------------------------------------|
| 1 | Sensores prontos + RPi inicia handshake | RPi → STM32 | [START]<br>[SYS_RDY]   | AA 10       | Raspberry Pi sinaliza que está pronto. STM32 só aceita após todos os sensores medirem 200 Hz. |
| 2 | STM32 confirma inicialização            | STM32 → RPi | [CMD_OK]<br>[SYS_INIT] | 90 01       | STM32 confirma sistema inicializado.                                                          |
| 3 | STM32 informa modo inicial              | STM32 → RPi | [CMD_OK]<br>[MODE_FSM] | 90 11       | STM32 informa que inicia em OP_MODE_FSM. Loop de boot encerra. FSM inicia em STATE_IDLE.      |

Ciclo Completo — Objeto encaminhado para Rota A

| # | Evento / Estado FSM                       | Agente      | Quadro Completo            | Bytes (hex) | Descrição                                                                               |
|---|-------------------------------------------|-------------|----------------------------|-------------|-----------------------------------------------------------------------------------------|
| 1 | IDLE → Sensor 1 detecta objeto            | STM32 → RPi | [CMD_OK]<br>[OBJ_DETECTED] | 90 A0       | STM32 notifica detecção. FSM avança para STATE_OBJECT_DETECTED.                         |
| 2 | OBJECT_DETECTED → Sensor 2 detecta objeto | STM32 → RPi | [CMD_OK]<br>[CLSS_REQUEST] | 90 C0       | STM32 para o motor e solicita classificação. FSM avança para STATE_WAIT_CLASSIFICATION. |
| 3 | RPi define destino: Rota A                | RPi → STM32 | [START]<br>[ROUTE_A]       | AA DA       | Raspberry Pi envia o resultado da classificação por visão computacional.                |
| 4 | STM32 confirma e encaminha para Rota A    | STM32 → RPi | [CMD_OK]<br>[ROUTE_A_FWD]  | 90 FA       | STM32 confirma, aciona servo (closeAngle), religa motor. FSM avança para STATE_ROUTE_A. |
| 5 | ROUTE_A → Sensor 3 detecta objeto         | STM32 → RPi | [CMD_OK]<br>[ROUTE_A_OK]   | 90 BA       | Entrega confirmada na Rota A. FSM retorna para STATE_IDLE.                              |

Ciclo Completo — Objeto encaminhado para Rota B

| # | Evento / Estado FSM                       | Agente      | Quadro Completo            | Bytes (hex) | Descrição                                                                               |
|---|-------------------------------------------|-------------|----------------------------|-------------|-----------------------------------------------------------------------------------------|
| 1 | IDLE → Sensor 1 detecta objeto            | STM32 → RPi | [CMD_OK]<br>[OBJ_DETECTED] | 90 A0       | STM32 notifica detecção. FSM avança para STATE_OBJECT_DETECTED.                         |
| 2 | OBJECT_DETECTED → Sensor 2 detecta objeto | STM32 → RPi | [CMD_OK]<br>[CLSS_REQUEST] | 90 C0       | STM32 para o motor e solicita classificação. FSM avança para STATE_WAIT_CLASSIFICATION. |
| 3 | RPi define destino: Rota B                | RPi → STM32 | [START]<br>[ROUTE_B]       | AA DB       | Raspberry Pi envia o resultado da classificação por visão computacional.                |
| 4 | STM32 confirma e encaminha para Rota      | STM32 → RPi | [CMD_OK]<br>[ROUTE_B_FWD]  | 90 FB       | STM32 confirma, aciona servo (openAngle), religa motor. FSM avança                      |

| # | Evento / Estado FSM                            | Agente      | Quadro Completo                                    | Bytes (hex)        | Descrição                                                                |
|---|------------------------------------------------|-------------|----------------------------------------------------|--------------------|--------------------------------------------------------------------------|
|   | B                                              |             |                                                    |                    | para <code>STATE_ROUTE_B</code> .                                        |
| 5 | <code>ROUTE_B</code> → Sensor 4 detecta objeto | STM32 → RPi | <code>[CMD_OK]</code><br><code>[ROUTE_B_OK]</code> | <code>90 BB</code> | Entrega confirmada na Rota B. FSM retorna para <code>STATE_IDLE</code> . |

Troca de Modo (qualquer estado)

| # | Evento                             | Agente      | Quadro Completo                                      | Bytes (hex)        | Descrição                                                                              |
|---|------------------------------------|-------------|------------------------------------------------------|--------------------|----------------------------------------------------------------------------------------|
| 1 | RPi solicita entrada em modo debug | RPi → STM32 | <code>[START]</code><br><code>[DEBUG_TOGGLE]</code>  | <code>AA DD</code> | Pode ser enviado em qualquer ponto, em qualquer estado da FSM.                         |
| 2 | STM32 ecoa o comando recebido      | STM32 → RPi | <code>[CMD_OK]</code><br><code>[DEBUG_TOGGLE]</code> | <code>90 DD</code> | Confirmação imediata do recebimento do frame.                                          |
| 3 | STM32 informa modo ativo           | STM32 → RPi | <code>[CMD_OK]</code><br><code>[MODE_DEBUG]</code>   | <code>90 22</code> | Motor travado ( <code>stepperStopEngaged</code> ), flash apagado. LED pisca 6× rápido. |
| 4 | RPi solicita retorno ao modo FSM   | RPi → STM32 | <code>[START]</code><br><code>[DEBUG_TOGGLE]</code>  | <code>AA DD</code> | Mesmo comando, reenviado para alternar de volta.                                       |
| 5 | STM32 ecoa o comando recebido      | STM32 → RPi | <code>[CMD_OK]</code><br><code>[DEBUG_TOGGLE]</code> | <code>90 DD</code> | Confirmação imediata do recebimento do frame.                                          |
| 6 | STM32 informa modo ativo           | STM32 → RPi | <code>[CMD_OK]</code><br><code>[MODE_FSM]</code>     | <code>90 11</code> | FSM reinicia em <code>STATE_IDLE</code> . LED acende 400 ms e apaga.                   |

Reset por Software (qualquer estado, qualquer modo)

| # | Evento                    | Agente      | Quadro Completo                                  | Bytes (hex)        | Descrição                                                                                                     |
|---|---------------------------|-------------|--------------------------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------|
| 1 | RPi solicita reset        | RPi → STM32 | <code>[START]</code><br><code>[SW_RESET]</code>  | <code>AA 33</code> | Processado com prioridade máxima, antes do toggle de modo e da FSM.                                           |
| 2 | STM32 confirma e reinicia | STM32 → RPi | <code>[CMD_OK]</code><br><code>[SW_RESET]</code> | <code>90 33</code> | STM32 envia confirmação, aguarda ~50 ms para o frame sair pelo TX e executa <code>NVIC_SystemReset()</code> . |

Modo Debug — `OP_MODE_DEBUG`

Em modo debug a FSM fica suspensa. O STM32 responde exclusivamente a comandos assíncronos e envia telemetria de sensores espontaneamente.

Telemetria de Sensores (espontânea)

| # | Evento                        | Agente      | Quadro Completo                            | Bytes (hex) | Descrição                                                                                                                                                                                                     |
|---|-------------------------------|-------------|--------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | STATUS_BYTE dos sensores muda | STM32 → RPi | [CMD_OK]<br>[SENS_STATUS]<br>[STATUS_BYTE] | 90 55<br>NN | Enviado apenas quando o valor do STATUS_BYTE muda em relação ao anterior.<br>NN: bit 0 = SENS1, bit 1 = SENS2, bit 2 = SENS3, bit 3 = SENS4 (1 = objeto detectado, 0 = feixe livre). Bits 4–7 reservados (0). |

### Comandos Assíncronos

| #  | Comando enviado             | Agente      | Quadro Completo                 | Bytes (hex)    | Ação executada pelo STM32                              | Resposta STM32           | Bytes (hex) |
|----|-----------------------------|-------------|---------------------------------|----------------|--------------------------------------------------------|--------------------------|-------------|
| 1  | Liga luminária              | RPi → STM32 | [START]<br>[LIGHT_EN]           | AA<br>E1       | FLASH_PIN → HIGH                                       | [CMD_OK]<br>[LIGHT_EN]   | 90<br>E1    |
| 2  | Desliga luminária           | RPi → STM32 | [START]<br>[LIGHT_DIS]          | AA<br>D1       | FLASH_PIN → LOW                                        | [CMD_OK]<br>[LIGHT_DIS]  | 90<br>D1    |
| 3  | Abre cancela                | RPi → STM32 | [START]<br>[GATE_OPEN]          | AA<br>E2       | Servo → <code>openAngle</code> (45°)                   | [CMD_OK]<br>[GATE_OPEN]  | 90<br>E2    |
| 4  | Fecha cancela               | RPi → STM32 | [START]<br>[GATE_CLOSE]         | AA<br>D2       | Servo → <code>closeAngle</code> (0°)                   | [CMD_OK]<br>[GATE_CLOSE] | 90<br>D2    |
| 5  | Ativa motor (eixo engajado) | RPi → STM32 | [START]<br>[STPR_EN]            | AA<br>E3       | <code>stepperFollowSteps()</code>                      | [CMD_OK]<br>[STPR_EN]    | 90<br>E3    |
| 6  | Desativa motor (eixo livre) | RPi → STM32 | [START]<br>[STPR_DIS]           | AA<br>D3       | <code>stepperStopDisengaged()</code>                   | [CMD_OK]<br>[STPR_DIS]   | 90<br>D3    |
| 7  | Define direção: frente      | RPi → STM32 | [START]<br>[STPR_FWD]           | AA<br>E4       | <code>stepperDirInst = 0</code>                        | [CMD_OK]<br>[STPR_FWD]   | 90<br>E4    |
| 8  | Define direção: trás        | RPi → STM32 | [START]<br>[STPR_BCK]           | AA<br>D4       | <code>stepperDirInst = 1</code>                        | [CMD_OK]<br>[STPR_BCK]   | 90<br>D4    |
| 9  | Define quantidade de passos | RPi → STM32 | [START]<br>[STPR_TGT]<br>[DATA] | AA<br>E5<br>NN | <code>targetStepps = DATA</code> (NN = valor de 0–255) | [CMD_OK]<br>[STPR_TGT]   | 90<br>E5    |
| 10 | Comando desconhecido        | RPi → STM32 | [START][???                     | AA<br>??       | Nenhuma ação                                           | [CMD_ERR]                | 91          |